

12. RTL · 디지털 설계 기초 — "나는 RTL을 직접 설계할 수 있다"

🔥 이 문서의 목적은 둘이다. 첫째, RTL 디지털 설계의 1st-principle을 밑바닥부터 다시 세워 면접의 어떤 꼬리질문도 "원리"로 답하게 한다. 둘째 — 그리고 이쪽이 더 중요한데 — "방법론·검증만 했지 직접 RTL을 짤 수 있나?"라는 의심을 정면으로 무너뜨린다. 나는 학부에서 5-stage MIPS 파이프라인 프로세서, SoC, VLSI를 직접 설계했고, SRAM을 주도적으로 설계했으며, AES 암호 모듈을 RTL로 구현해 S-box를 256B LUT 대신 logic gate로 재구현하여 27,000→10,000 GE로 63% 줄였다(04_resume 9번). 검증/sign-off를 4년 한 사람이 RTL을 못 짜는 게 아니라, RTL을 깊이 이해했기 때문에 sign-off를 잘하는 것이다. 합성 후 무엇이 깨지는지 아는 사람이 곧 안 깨지는 RTL을 짤다.

🧠 메모리 브릿지: 메모리 컨트롤러의 command FSM·refresh 카운터·burst 파이프라인, 다중 reset 도메인, Peri의 SRAM 컴파일러 사용이 전부 본 문서의 내용이다.

연결: 메타스터빌리티·CDC·RDC·reset 동기화의 깊은 이야기는 05_CDC_RDC_완전이해.md, setup/hold·recovery/removal·타이밍 수식은 06_STA_완전이해.md에서 다룬다. 본 문서는 그 둘과 겹치는 부분은 "자세히는 05/06 참조"로 연결하고 RTL 설계 자체에 집중한다.

0. 큰 그림 — 이 문서를 읽는 지도

RTL 디지털 설계는 한 문장으로 "원하는 동작을, 레지스터와 레지스터 사이의 조합논리로 기술하는 일"이다. 본 문서는 이 한 문장을 다음 순서로 펼친다. ① RTL이 무엇인가 → ② 조합 vs 순차, 동기 설계 규율 → ③ Verilog/SystemVerilog 핵심(blocking/non-blocking, latch, sim-synth mismatch) → ④ FSM(Moore/Mealy) → ⑤ 파이프라이닝 → ⑥ 해저드 → ⑦ Reset 전략 → ⑧ Glitch와 금기 → ⑨ 자주 묻는 빌딩블록 → ⑩ 합성 → 게이트→PPA, 메모리 컴파일러 → ⑪ 면접 골격 → ⑫ 스스로 점검.

읽는 자세: 코드를 외우지 말고 "왜 이 코드가 이 게이트로 합성되는가"를 머릿속에 그려라. RTL 설계자의 머릿속에는 항상 코드가 아니라 회로가 있다. 코드는 그 회로를 적는 한 가지 표기일 뿐이다.

1. RTL 설계란 무엇인가 — 동작을 레지스터 사이 조합논리로 기술한다

1.1 추상화 레벨의 사다리

디지털 설계는 추상화 사다리로 표현할 수 있다. 위에서 아래로:

- 알고리즘/동작(behavioral): "두 수를 더해 누적한다" 같은 의도.
- RTL(Register Transfer Level): "매 클럭마다 레지스터 A에 $(A + B)$ 를 저장한다." — 데이터가 어느 레지스터에서 어느 레지스터로, 그 사이에 어떤 조합논리를 거쳐 이동하는지를 기술한다. 이름 그대로 *Register Transfer*다.

- **게이트(gate netlist)**: AND/OR/FF 같은 표준셀의 연결망. 합성 도구가 RTL로부터 만든다.
- **트랜지스터/레이아웃**: 물리적 구현.

설계자가 사람 손으로 짜는 가장 추상적인 레벨이 바로 RTL이다. RTL보다 위(HLS)는 도구가 더 많이 결정하고, 아래(게이트)는 도구가 거의 다 한다. **RTL은 "사람이 회로 구조를 통제하는 마지막 레벨"**이라서, RTL을 짤 줄 안다는 것은 곧 "최종 게이트가 어떤 모양일지 머릿속에 그릴 줄 안다"는 뜻이다.

1.2 핵심 멘탈 모델 — "레지스터 사이의 구름"

RTL을 떠올릴 때 가장 유용한 그림은 이것이다.



레지스터(플립플롭, FF)가 클럭 엣지마다 값을 "찰칵" 붙잡는 **댐**이라면, 그 사이의 조합논리는 데이터가 흘러가며 변형되는 **물길(구름)**이다. 매 클럭 엣지에 모든 댐이 동시에 열리고, 한 주기 동안 물(데이터)이 다음 댐까지 도달해 안정되어야 한다. 도달 못 하면 **setup 위반**(타이밍 fail). 이 그림 하나가 STA, 파이프라이닝, 해저드, glitch를 전부 설명한다.

RTL을 짠다는 것은 결국 두 가지를 정하는 일이다. (1) **어떤 레지스터들을 둘 것인가(상태/저장)**, (2) **그 사이 조합논리를 어떻게 짤 것인가(연산)**. 그게 전부다. 복잡해 보이는 모든 설계가 이 두 결정의 반복이다.

1.3 코드 ↔ 회로 대응 (가장 중요한 직관)

같은 동작을 코드로 적으면 머릿속 회로가 떠올라야 한다. 예:

```
always_ff @(posedge clk) begin
    acc <= acc + data; // 레지스터 acc, 그 앞에 덧셈기, 출력이 acc로 되먹임
end
```

이 5줄을 보면 머릿속에 "**acc 레지스터 + 덧셈기 + acc→덧셈기로 가는 피드백 배선**"이 그려져야 한다. 반대로 "누산기"를 그리라고 하면 위 코드가 나와야 한다. 이 양방향 대응이 안 되면 그건 "코드를 적은 것"이지 "설계한 것"이 아니다. 면접에서 회로를 칠판에 그리라 하거나 코드를 보고 무슨 회로나 물을 때, 이 대응을 즉석에서 해내는 게 핵심이다.

갭 방어 톤: 검증/sign-off는 "남이 짠 RTL이 어떤 게이트가 되는지"를 끝까지 추적하는 일이다. LEC(논리등가검증, 04_resume 4번)는 RTL과 게이트가 같은 회로인지 수학적으로 증명한다. 즉 나는 4년간 "코드↔회로 대응"을 게이트 레벨까지 검증해왔다. 직접 짜는 사람보다 오히려 더 깊이 본 셈이다.

2. 조합 논리 vs 순차 논리, 그리고 동기 설계의 규율

2.1 조합 논리 — 메모리가 없다

조합(combinational) 논리는 **현재 입력만으로 출력이 즉시 결정**된다. 과거를 기억하지 않는다. AND, OR, MUX, 덧셈기, 디코더, 비교기가 모두 조합이다. 입력이 바뀌면 (게이트 지연 후) 출력이 따라 바뀐다.

```
// 조합 논리 — 메모리 없음, 입력의 함수
always_comb begin
    if (sel) y = a;
    else    y = b;          // 2:1 MUX
end
// 또는 더 간결히: assign y = sel ? a : b;
```

조합 논리의 위험은 **glitch**(8장)와 **지연**(critical path)이다. 출력은 가장 늦게 도착하는 입력 경로 + 게이트 지연만큼 늦어진다.

2.2 순차 논리 — 상태를 기억한다

순차(sequential) 논리는 **과거(상태)를 저장**한다. 저장 소자가 플립플롭(FF)이다. FF는 클럭 엣지(보통 상승엣지)에서만 입력 D를 받아 출력 Q로 내보낸다. 엣지 사이에는 값을 유지한다.

```
// 순차 논리 — 클럭 엣지에 상태 갱신
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) q <= 1'b0;    // 비동기 리셋
    else        q <= d;       // 엣지에서 d를 캡처
end
```

FF가 곧 1장의 "담"이다. 순차 논리 = 조합 논리(구름) + FF(담)의 조합으로 만들어진다. **모든 디지털 회로는 결국 FF와 그 사이 조합논리의 묶음**이다.

2.3 왜 동기 설계가 주류인가 — STA로 정적 검증이 가능하기 때문

동기(synchronous) 설계의 규율: 칩 안 거의 모든 FF가 **같은 클럭(또는 잘 정의된 소수의 클럭)**을 받는다. 그러면 "한 클럭 주기 안에, 모든 FF→조합논리→다음 FF 경로가 안정되어야 한다"는 단 하나의 규칙으로 전체 타이밍을 기술할 수 있다. 이 규칙을 입력 벡터 없이 **모든 경로에 대해 정적으로** 검사하는 게 STA(Static Timing Analysis)다.

비동기(asynchronous) 설계는 글로벌 클럭이 없어 잠재적으로 더 빠르고 저전력이지만, race-hazard가 많고 "언제 무엇이 안정되는가"를 정적으로 검증할 도구가 약하다. 그래서 상용 ASIC/메모리는 거의 전부 동기 설계이고, **클럭 도메인 경계만 비동기로 남겨 CDC로 다룬다**(자세히는 05 참조). 동기 설계가 이긴 이유는 "빠르다"가 아니라 **"검증 가능하다(predictable)"**이다. 이 한 줄이 면접 단골 Q1의 핵심이다.

메모리 브릿지: 메모리 IP는 코어 로직 도메인, PHY/IO 도메인, 관리(JTAG/refresh) 도메인 등 여러 동기 도메인의 조합이고, 그 경계가 CDC/RDC다. "동기 섬들 + 비동기 다리"가 칩의 일반적 구조다.

3. Verilog/SystemVerilog 핵심 — 합성 가능한 RTL의 규율

여기가 "직접 짤 수 있나"를 가장 직접적으로 증명하는 장이다. 합성 가능한(synthesizable) RTL에는 몇 가지 철칙이 있고, 이를 어기면 시뮬은 통과하는데 칩이 다르게 동작한다.

3.1 세 가지 always 블록 — always_ff / always_comb / latch

SystemVerilog는 의도를 명시적으로 적게 해준다(구식 Verilog의 모호한 `always @(*)` 보다 안전).

- `always_ff` : 플립플롭(엣지 트리거 순차논리)을 만들겠다는 선언. 클럭/리셋 sensitivity를 명시.
- `always_comb` : 순수 조합논리를 만들겠다는 선언. sensitivity list를 도구가 자동·완전하게 잡아준다(누락 불가).
- `latch` : 보통 **의도치 않게** 생긴다(아래 3.4). 명시적 래치(`always_latch`)는 클럭 게이팅 같은 특수 경우 외엔 피한다.

```
// FF: 상태 저장
always_ff @(posedge clk) q <= d;

// 조합: 즉시 함수
always_comb y = a & b;
```

원칙: 순차는 `always_ff`, 조합은 `always_comb` 로 명확히 분리한다. 한 블록에서 둘을 섞지 않는다. 메모리 컨트롤러처럼 큰 FSM도 "다음상태 계산(comb) + 상태 레지스터(ff)"를 두 블록으로 나누는 게 표준이다.

3.2 blocking(=) vs non-blocking(<=) — 가장 중요한 구분

이건 면접 단골이자 실수의 원흉이다. 정확히 이해해야 한다.

- **non-blocking <=** : 우변을 **현재 시점 값으로 모두 평가한 뒤**, 그 클럭 엣지가 끝날 때 **동시에** 좌변에 대입한다. 즉 같은 블록 안 여러 <= 는 병렬로 일어난다. → 순차 논리(FF)에 쓴다.
- **blocking =** : 즉시 평가·즉시 대입하고, 다음 문장으로 넘어간다. 위에서 아래로 **순차적으로** 실행된다. → 조합 논리에 쓴다.

왜 FF에 <= 를 써야 하는가? 시프트 레지스터로 보면 즉시 이해된다.

```
// 올바른: 3단 시프트. <=는 우변을 동시에 평가 → 진짜 시프트
always_ff @(posedge clk) begin
    q1 <= d;
    q2 <= q1;    // 여기서 q1은 "이번 엣지 직전의" q1
    q3 <= q2;
end
```

만약 여기서 = (blocking)를 쓰면 `q1 = d; q2 = q1;` 이 순차 실행되어 q2가 새 q1(=d)을 받아버린다 → 3단 시프트가 아니라 **모두 d가 되는** 잘못된 회로. 반대로 조합논리에서 <= 를 쓰면 중간값이 한 박자 늦게 반영되어 의도와 다른 결과가 난다.

race condition: 두 always 블록이 같은 신호를 한쪽은 읽고 한쪽은 쓰는데 둘 다 blocking을 쓰면, 시뮬레이터의 실행 순서에 따라 결과가 달라지는 race가 생긴다. non-blocking 규율(FF는 <=)은 이 race를 원천 차단한다 —

모든 FF가 "엣지 끝에 동시에" 갱신되므로 순서 의존성이 사라진다.

철칙(외워라): 순차(클럭드) 블록 = <= 만, 조합 블록 = = 만. 이 둘을 섞지 않는다. 메모리에서 burst 데이터 시프트, command 파이프라인 모두 이 규율 위에 선다.

3.3 sensitivity list — 무엇이 바뀌면 다시 계산하나

조합 always 블록은 "입력 중 하나라도 바뀌면 출력을 재계산"해야 한다. 구식 Verilog에서는 그 입력 목록 (sensitivity list)을 손으로 적었는데, 빠뜨리면(incomplete sensitivity list) **시물은 빠뜨린 입력에 반응 안 하고, 합성은 전체 입력으로 회로를 만든다** → 둘이 갈린다(sim-synth mismatch). SystemVerilog의 always_comb 는 도구가 모든 우변 신호를 자동으로 sensitivity에 넣어주므로 이 실수를 구조적으로 막는다. 그래서 신규 RTL은 always @(a or b) 대신 always_comb 를 쓰는 게 합성 가이드의 기본이다.

3.4 의도치 않은 latch 추론 — 모든 경로에 대입하라

조합 always 블록에서 **어떤 입력 조합에서 출력에 대입이 누락되면**, 도구는 "그 경우엔 출력이 이전 값을 유지해야 한다"고 해석해 래치를 만든다. 래치는 레벨 민감이라 동기 설계에 끼면 타이밍·glitch 문제를 일으킨다.

```
// 나쁨: else가 없어 sel=0일 때 y 미대입 → latch 추론
always_comb begin
    if (sel) y = a;
    // else 누락!
end

// 좋음 1: 모든 경로에 대입
always_comb begin
    if (sel) y = a;
    else    y = b;
end

// 좋음 2: 기본값 먼저 (case에서 특히 유용)
always_comb begin
    y = 1'b0;          // default 먼저
    if (sel) y = a;
end
```

규칙: 조합 블록은 모든 분기에서 모든 출력에 대입하라. case는 default 를, if는 else 를, 혹은 블록 맨 위에서 모든 출력에 기본값을 주는 패턴으로 latch를 원천 차단한다. lint 도구가 잡는 대표 위반이 바로 "inferred latch"다 (내가 sign-off에서 늘 보던 것).

3.5 simulation-synthesis mismatch — "왜 시물 통과 RTL이 합성 후 달라지나"

이건 04_resume 5번, G_digital Q5와 직결되는 면접 핵심이다. 시물(동작 기술)과 합성(게이트 매핑)이 갈리는 대표 원인:

1. incomplete sensitivity list (3.3) — 시물은 일부 입력에만 반응, 합성은 전체로 봄.
2. 의도치 않은 latch 추론 (3.4) — if/case 분기 대입 누락.
3. blocking/non-blocking 오용 (3.2) — 시물 실행 순서 의존 race.

4. **비합성 구문 사용** — `initial`, `#delay`, `$display`, 실수 등은 시뮬에선 동작하지만 합성은 무시·에러. 합성 가능 RTL은 이런 구문을 디자인 로직에 쓰지 않는다(테스트벤치에만).
5. **x / z 처리 차이** — 시뮬의 X-propagation과 합성 게이트(0/1만)의 차이로 X-optimism/pessimism 발생.
6. **부주의한 함수/연산** — 부호 확장, 비트폭 truncation 등.

이 mismatch를 막는 sign-off 단계가 **lint(코딩 규칙 검사) + CDC + LEC(논리등가검증)**다. 즉 "RTL을 짜는 능력"과 "RTL이 합성 후 안 깨지게 하는 능력"은 동전의 양면이고, 후자가 바로 내 4년 전문이다.

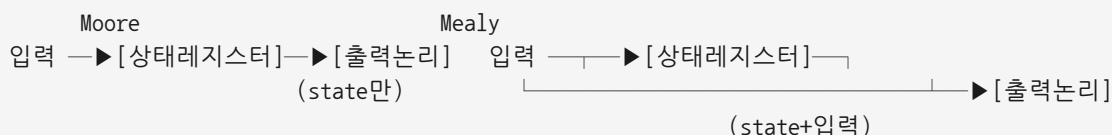
갭 방어 톤(강조): 면접관이 "방법론만 한 거 아니냐"고 물으면 — "오히려 반대입니다. sim-synth mismatch가 왜 생기는지(sensitivity·latch·blocking·X) 게이트 레벨까지 추적해온 사람이라, 처음부터 그게 안 생기는 합성 친화 RTL을 짭니다. LEC로 RTL=netlist를 수천만 line 규모로 sign-off한 경험이 곧 '내가 짠 RTL이 의도대로 게이트가 된다'는 확신의 근거입니다."

4. FSM 완전이해 — Moore vs Mealy, encoding, 안전한 설계

상태 기계(FSM, Finite State Machine)는 "현재 상태 + 입력 → 다음 상태 + 출력"을 정의한다. 메모리 command 시퀀서(ACT→RD/WR→PRE), 버스 프로토콜, 핸드셰이크, refresh 제어가 전부 FSM이다.

4.1 Moore vs Mealy — 출력 의존성의 차이

- **Moore FSM:** 출력이 **현재 상태에만** 의존. 출력이 상태 레지스터의 함수라 **registered**처럼 안정적, **glitch 없음**. 단, 입력 변화가 출력에 반영되려면 상태가 먼저 바뀌어야 하므로 **1 클럭 latency**가 더 든다.
- **Mealy FSM:** 출력이 **상태 + 현재 입력** 모두에 의존. 입력에 **즉각 반응**하고 보통 **상태 수가 더 적다**. 그러나 입력의 조합 경로가 출력에 직접 연결돼 **입력 glitch가 출력으로 전파**되고 타이밍이 까다롭다.



절충 = registered Mealy: Mealy 출력을 FF 한 단으로 다시 받아 glitch를 없애고 타이밍을 깔끔히 한다. Mealy의 "적은 상태"와 Moore의 "안정 출력"을 합친다. 실무에서 자주 쓴다.

면접 골격(Q): "Moore는 출력이 상태만의 함수라 glitch 없이 안정적이거나 1클럭 느리고, Mealy는 상태+입력이라 빠르고 상태가 적지만 입력 glitch가 출력에 셉니다. 보통 출력을 한 번 더 register해 둘을 절충합니다."

4.2 state encoding — binary / one-hot / gray

상태를 비트로 어떻게 인코딩하느냐의 trade-off:

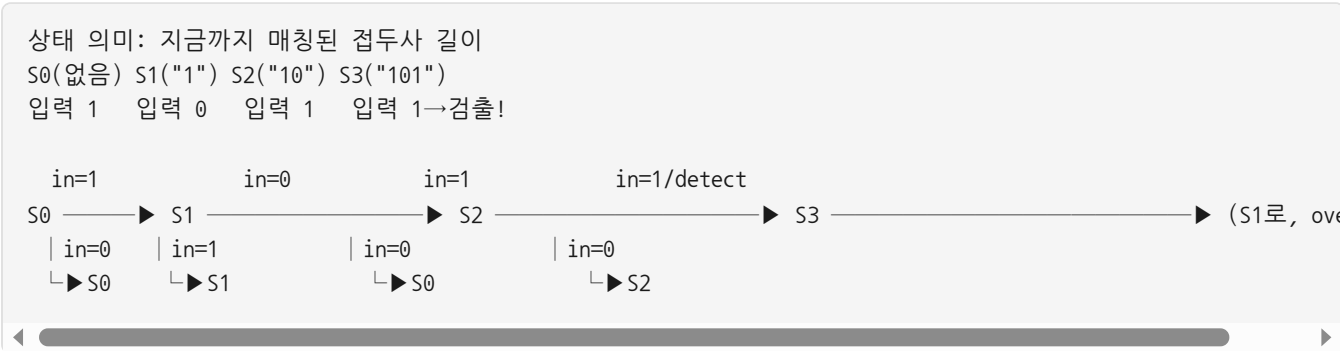
- **binary:** N 상태에 $\lceil \log_2 N \rceil$ 비트. **FF 최소**, 그러나 다음상태·출력 디코딩 논리가 깊어질 수 있음. 면적이 중요할 때.
- **one-hot:** 상태마다 1비트(N 상태=N FF). FF는 많지만 **디코딩이 단순·빠름**(상태 판별 = 한 비트 검사). FPGA·고속 설계에서 선호.

- **gray**: 인접 상태 전이 시 **1비트만 변함**. 다중 비트 동시 천이가 없어 **저전력·glitch 적음**, 특히 CDC를 건너는 카운터(async FIFO 포인터)에 필수(자세히는 O5 참조).

요약: 면적이면 binary, 속도/단순함이면 one-hot, 도메인 횡단/저전력이면 gray.

4.3 작은 예시 — "1011" 시퀀스 검출기(overlap 허용, Mealy)

직렬 입력 비트 스트림에서 "1011" 패턴이 나타나면 detect=1. overlap 허용(예: ...1011011... 에서 두 번 검출). 상태도:



(overlap: "1011"의 끝 "1"이 다음 패턴의 시작 "1"이 될 수 있으므로 검출 후 S1로, S3에서 in=0이면 끝 "1...0"의 "10"이 살아 S2로 간다.)

코드(2-block + Mealy 출력):

```
typedef enum logic [1:0] {S0, S1, S2, S3} state_t;
state_t state, next;

// 1) 상태 레지스터 (순차, <=)
always_ff @(posedge clk or negedge rst_n)
    if (!rst_n) state <= S0;
    else      state <= next;

// 2) 다음상태 논리 (조합, =, 모든 경로 대입)
always_comb begin
    next = state;          // 기본값 → latch 방지
    unique case (state)
        S0: next = in ? S1 : S0;
        S1: next = in ? S2 : S0;
        S2: next = in ? S3 : S0;
        S3: next = in ? S1 : S2;
    endcase
end

// 3) Mealy 출력: 상태 S3 + 입력 1 → 검출
assign detect = (state == S3) && in;
```

여기 3장의 규율이 전부 들어있다: `always_ff` 엔 `<=`, `always_comb` 엔 `=` +기본값 대입(latch 방지), `enum`으로 상태 가독성. Moore 버전이면 detect를 상태(예: 별도 ACCEPT 상태)만으로 내고 한 클럭 늦게 나온다.

4.4 안전한 FSM — 기본 상태 복구

실제 칩에서는 SEU(soft error)나 미사용 인코딩 진입으로 FSM이 정의되지 않은 상태에 빠질 수 있다(특히 one-hot은 사용 안 하는 조합이 많다). **안전한 FSM**은 case에 default를 두어 알 수 없는 상태에서 안전한 상태(보통 IDLE)로 복구하게 한다. 합성 도구의 "safe FSM" 옵션도 이 복구 논리를 자동 삽입한다.

```
always_comb begin
    next = IDLE;           // 알 수 없는 상태면 IDLE로 복구
    unique case (state)
        ... // 정의된 전이
        default: next = IDLE;
    endcase
end
```

메모리 브릿지: DRAM command FSM은 ACT→(tRCD 대기)→RD/WR→(tRAS 충족)→PRE 순으로 가고, 각 전이는 **타이밍 카운터**(다음 9장 카운터)와 결합된다. refresh 제어도 주기 카운터 + FSM이다. FSM+카운터 결합이 메모리 컨트롤러 RTL의 뼈대다.

5. 파이프라이닝 — throughput를 올리고 latency는 유지/증가

5.1 기본 아이디어

긴 조합 경로 하나를 레지스터로 여러 **stage**로 쪼갬다. 각 stage가 짧아지면 한 주기에 필요한 시간이 줄어 **클럭 주파수(=throughput)**를 올릴 수 있다. 같은 일을 더 빠른 클럭으로 처리하는 것이다.

파이프라인 전:	[FF]—(긴 조합: 10ns)—[FF]	→ $f_{max} \approx 100\text{MHz}$
파이프라인 후:	[FF]—(5ns)—[FF]—(5ns)—[FF]	→ $f_{max} \approx 200\text{MHz}$ (대략)

핵심 구분(면접에서 자주 헷갈림): - **throughput(처리량)**: 단위시간당 완료되는 결과 수 → **올라간다**(주파수↑). - **latency(지연)**: 하나의 데이터가 입력에서 출력까지 걸리는 시간 → **stage 수만큼 클럭이 늘어 유지되거나 증가한다**.

비유: 세탁(빨래→건조→개기)을 한 사람이 끝까지 하면 한 벌에 90분(latency), 시간당 한 벌(throughput). 3단 파이프(빨래기·건조기·개는 사람 분리)면 한 벌은 여전히 90분이지만, 정상 가동 시 **30분마다 한 벌**이 나온다(throughput 3배). 한 벌의 시간은 안 줄고 **흐름이 빨라진다**.

5.2 stage 균형과 retiming

파이프라인의 f_{max} 는 **가장 긴 stage**가 결정한다(병목). 그래서 stage들을 **균형** 있게 쪼개는 게 핵심이다. 한 stage가 6ns, 다른 게 4ns면 클럭은 6ns에 묶인다. 합성 도구의 **retiming**은 기능을 유지한 채 레지스터를 조합논리 사이로 옮겨 stage 시간을 자동 균형화한다(예: 6/4 → 5/5). (구체적 타이밍 마진·slack 계산은 06 참조)

5.3 왜 무한정 깊게 못 쪼개나

stage를 늘릴수록 좋아 보이지만 한계가 있다:

- **레지스터 오버헤드**: 매 stage 경계마다 FF의 $clk \rightarrow Q$ 지연 + setup 시간 이 더해진다. stage 조합논리가 이 오버헤드보다 작아지면 더 쪼개도 주파수 이득이 사라진다.
- **latency·면적·전력 증가**: stage가 늘면 FF가 늘어 면적·클럭 전력 $\uparrow$, latency(클럭 수) $\uparrow$.
- **해저드 penalty 증가**: 파이프가 깊을수록 분기 오예측·데이터 의존(다음 6장)의 비용이 커진다. 그래서 옛 초고주파 CPU의 초깊은 파이프가 오히려 비효율이었다.

결론: **최적 파이프 깊이가 있다**. "더 쪼개면 빨라지냐"는 꼬리질문엔 "오버헤드·latency·해저드 penalty 때문에 최적점이 있다"가 정답.

🧠 메모리: 메모리 데이터패스(ECC encode/decode, burst 정렬), PHY의 직병렬 변환이 모두 파이프라인으로 대역폭을 확보한다. 고대역 HBM/GDDR일수록 깊은 파이프와 stage 균형이 중요하다.

6. 해저드 — 파이프라인이 만드는 부작용 (MIPS 경험 연결)

파이프라인은 공짜가 아니다. 여러 명령(또는 트랜잭션)이 stage에 겹쳐 흐르며 충돌이 생긴다. 이를 **해저드(hazard)**라 한다. 나는 학부에서 **5-stage MIPS 파이프라인 프로세서를 직접 설계**(04_resume 자소서 근거)하며 이 셋을 직접 풀었다.

6.1 세 종류

1. **구조적 해저드(structural)**: 같은 하드웨어 자원을 동시에 두 stage가 쓰려는 충돌. 예: 명령 fetch와 데이터 access가 단일 메모리 포트를 다툼. → 자원 분리(Harvard, 명령/데이터 메모리 분리)나 stall로 해결.
2. **데이터 해저드(data)**: 앞 명령의 결과를 뒤 명령이 필요로 하는데 아직 안 나옴. - **RAW(Read After Write)**: 진짜(real) 의존. 앞이 쓴 값을 뒤가 읽어야 함. 파이프라인에서 실제로 막아야 하는 핵심. - **WAR(Write After Read)**, **WAW(Write After Write)**: in-order 단순 파이프에선 발생 안 하고, out-of-order/rename 환경의 이름(name) 의존. 그래서 "RAW만 real, WAR/WAW는 register renaming으로 제거 가능"이라 답한다.
3. **제어 해저드(control)**: 분기(branch)로 다음에 어느 명령을 가져올지 불확실. 분기 결과가 나오기 전까지 다음 fetch가 추측이 됨.

6.2 해결 기법

- **forwarding(bypass)**: 결과를 레지스터 파일에 쓰기 전에, 계산이 끝난 stage(예: EX/MEM 출력)에서 **직접 다음 명령의 입력으로 우회 배선**해 RAW를 stall 없이 해소. 가장 핵심적인 데이터 해저드 해결책.
- **stall(정지) / bubble(거품)**: forwarding으로도 못 푸는 경우(예: load 직후 사용, load-use hazard) 한 사이클 멈추고 빈 거품(NOP)을 끼워 의존을 기다린다.
- **branch prediction(분기 예측)**: 제어 해저드를 줄이려 분기 방향/타겟을 미리 추측해 fetch를 계속. 틀리면 (misprediction) 잘못 가져온 명령을 flush(파이프 비우기). 예측이 깊은 파이프일수록 중요.

면접 골격(Q): "해저드는 구조적·데이터(주로 RAW)·제어 셋입니다. 데이터는 forwarding으로 결과를 다음 stage에 우회시키고, 못 풀면 stall/bubble로 기다리며, 제어는 분기 예측으로 줄이고 틀리면 flush합니다. 저는 학부 5-stage MIPS에서 forwarding 유닛과 hazard detection을 직접 구현했습니다." ← 직접 설계 증거.

🧠 메모리: 메모리 컨트롤러도 read-after-write 위험(같은 주소의 쓰기 직후 읽기)을 hazard로 다뤄 forwarding 버퍼나 ordering 제약으로 푼다. 본질은 CPU 파이프와 같다.

7. Reset 전략 — sync vs async, 그리고 reset synchronizer

(reset이 도메인을 건너는 RDC, recovery/removal 타이밍의 깊은 내용은 05_CDC_RDC·06_STA 참조. 여기서 RTL 설계 관점에서 어떤 reset을 어떻게 코딩하나만.)

7.1 sync reset vs async reset

- **synchronous reset**: 클럭 엣지에서만 reset이 적용된다. `always_ff @(posedge clk)` 안에서 `if(rst) ...` 로 코딩. 장점: 타이밍이 깨끗(reset도 일반 데이터처럼 STA에 포함), glitch 내성. 단점: **running clock이 있어야 reset이 먹고**, reset 펄스 폭이 한 주기 이상이어야 한다.

```
// sync reset: sensitivity에 clk만
always_ff @(posedge clk)
  if (rst) q <= 1'b0;
  else    q <= d;
```

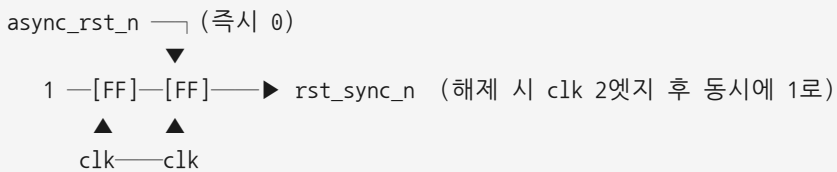
- **asynchronous reset**: 클럭과 무관하게 즉시 assert. `always_ff @(posedge clk or negedge rst_n)`. 장점: 클럭이 죽어도(전원 직후 등) 즉시 초기화 가능. 단점: **deassert(해제)가 클럭 엣지 근처에서 일어나면 recovery/removal 위반→메타스터빌리티**(자세히는 06 참조).

```
// async reset: sensitivity에 reset 엣지도
always_ff @(posedge clk or negedge rst_n)
  if (!rst_n) q <= 1'b0;
  else       q <= d;
```

7.2 reset synchronizer — 왜 필요한가 (async assert + sync deassert)

async reset의 강점(즉시 assert)은 살리고, 약점(해제 타이밍 메타)을 없애는 표준 회로가 **reset synchronizer**다. 핵심 원리: **assert는 비동기로 즉시**, **deassert(해제)는 클럭에 동기화**해서 모든 FF가 같은 클럭 엣지에 동시에 reset에서 풀리게 한다.

```
// 2-FF reset synchronizer: async assert, sync deassert
logic rff1, rff2;
always_ff @(posedge clk or negedge async_rst_n) begin
    if (!async_rst_n) begin
        rff1 <= 1'b0;          // async로 즉시 0 (assert)
        rff2 <= 1'b0;
    end else begin
        rff1 <= 1'b1;          // 해제는 clk 엣지로 단계적 전파 (sync deassert)
        rff2 <= rff1;
    end
end
assign rst_sync_n = rff2;    // 이 신호를 설계의 reset으로 사용
```



만약 reset synchronizer 없이 raw async reset을 그대로 풀면, FF마다 해제 시점이 미세하게 달라(skew) 어떤 FF는 풀리고 어떤 FF는 아직 reset인 **혼란 상태**나 메타가 생긴다. synchronizer는 "다 같이 assert, 다 같이(같은 엣지에) deassert"를 보장한다.

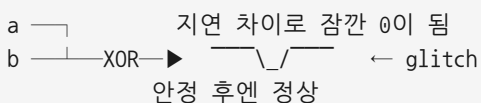
면접 골격(Q "reset synchronizer 왜?"): "async reset은 즉시 초기화엔 좋지만 해제가 클럭 엣지 근처면 recovery/removal 위반으로 메타가 납니다. 그래서 assert는 비동기로 즉시, deassert는 2-FF로 동기화해 모든 FF가 같은 엣지에 풀리게 하는 게 reset synchronizer입니다. reset도 신호라 도메인을 건너면 RDC로 CDC처럼 검증해야 합니다(제 RDC 매트릭스 방법론, 04_resume 1번)." ← **검증 경험과 설계가 한 줄로 연결.**

🧠 메모리: 메모리 IP는 power-on reset, soft reset, PHY reset 등 다중 reset의 **해제 정렬**이 까다롭다. 각 도메인에 reset synchronizer를 두고 도메인 간은 RDC로 sign-off한다.

8. Glitch와 금기 — 조합 출력을 어디에 쓰면 안 되는가

8.1 glitch란

조합 논리는 입력들이 **서로 다른 경로 지연**으로 도착해, 최종값으로 안정되기 전 잠깐 **잘못된 중간 천이**를 만든다. 이게 glitch(hazard)다.



8.2 핵심 금기 (외워라)

동기 FF의 **데이터 입력(D)**에 들어가는 glitch는 **무해**하다 — 클럭 엣지 시점엔 이미 안정값이라 그것만 캡처되기 때문. 그러나:

raw 조합 출력을 clock, reset, async set/clear, CDC enable처럼 "엣지/레벨로 직접 동작하는 입력"에 쓰면 절대 안 된다.

이유: 이런 입력은 glitch를 **진짜 이벤트로 오인**한다. 조합 신호를 클럭으로 쓰면 glitch가 가짜 클럭 엣지가 되어 FF가 엉뚱하게 토글하고, 조합 신호를 async reset에 쓰면 가짜 리셋이 걸린다. 그래서:

- 게이티드 클럭은 직접 AND로 만들지 않고 ICG(Integrated Clock Gating cell)로 만든다. ICG는 enable을 클럭 low 구간에 래치해 glitch-free 클럭을 보장한다.
- 클럭은 전용 클럭 트리(CTS)로만 분배하고, 일반 조합 신호를 클럭 네트워크에 섞지 않는다.
- glitch는 불필요한 토글로 **dynamic power**도 낭비한다 → path balancing으로 줄인다.

8.3 fan-out과 buffering

한 게이트가 너무 많은 부하(fan-out)를 구동하면 출력 transition(천이 시간)이 느려지고 지연이 커져 setup·신호 무결성이 나빠진다. 그래서 **buffer/inverter tree**로 부하를 나누고, 합성/물리 단계에서 **max fan-out·max transition** 제약을 둔다. 대표적 high fan-out net이 clock·reset·scan enable이고, 이들은 전용 트리(CTS)나 buffering으로 균형을 맞춘다.

면접 골격(Q "glitch 왜 위험?"): "조합 출력은 경로 지연차로 잠깐 잘못된 값(glitch)을 냅니다. FF의 D면 엣지에서 안정값만 잡혀 무해하지만, clock/reset/async/CDC enable에 raw 조합을 쓰면 가짜 이벤트로 오인돼 치명적입니다. 그래서 gated clock은 ICG로 만들고 조합 신호를 클럭으로 안 씁니다."

9. 자주 묻는 설계 빌딩블록 — 각각 핵심 아이디어 + 코드/구조

이 장은 "직접 짤 수 있다"를 손으로 증명하는 무기고다. 각 블록의 **핵심 아이디어**를 이해하면 즉석에서 변형해 짤 수 있다. (async FIFO·다중비트 동기화는 05 참조.)

9.1 카운터

핵심: FF + 덧셈기 + (wrap/enable/load) 제어. 모든 시퀀싱·타이밍 카운트(tRCD, refresh 주기)의 기초.

```
always_ff @(posedge clk or negedge rst_n)
  if (!rst_n)      cnt <= '0;
  else if (clear)  cnt <= '0;
  else if (en)     cnt <= (cnt == MAX) ? '0 : cnt + 1'b1; // wrap
```

9.2 클럭 분주기 ($\div 2$, $\div$ 홀수 50% duty)

- $\div 2$: 출력을 매 엣지 토글. 자동으로 50% duty. (단, 이렇게 만든 분주 클럭을 다른 로직의 클럭으로 직접 쓰면 클럭 트리/STA가 복잡해진다 — 가능하면 clock enable 방식 권장.)

```
always_ff @(posedge clk or negedge rst_n)
    if (!rst_n) clk_div2 <= 1'b0;
    else        clk_div2 <= ~clk_div2;    //  $\div 2$ , 50% duty
```

- $\div$ 짝수: 카운터로 $N/2$ 마다 토글 $\rightarrow$ 50% duty 쉬움.
- $\div$ 홀수에서 50% duty: 단순 카운터로는 듀티가 안 맞는다. 표준 트릭은 **posedge로 세는 분주와 negedge로 세는 분주를 OR**하는 것. 예를 들어 $\div 3$ 50% duty는 posedge 기반 펄스와 negedge 기반 펄스를 반 클럭 어긋나게 합쳐(OR) 만든다. 핵심 아이디어: "상승·하강 엣지 양쪽을 써서 반 클럭 해상도를 얻는다." (실무에선 이런 raw 분주 클럭보다 PLL/clock divider IP나 clock-enable을 쓴다.)

9.3 shift register

핵심: FF들을 직렬 연결, 매 클럭 한 칸 이동. 직병렬 변환(PHY 핵심), 지연선, LFSR(BIST 패턴/PRBS), 직렬 인터페이스의 기초.

```
always_ff @(posedge clk)
    shreg <= {shreg[N-2:0], serial_in};    // 좌측 시프트, MSB로 밀어넣기
// 직렬출력 = shreg[N-1]
```

(3.2의 $\leq$ 규율 덕에 진짜 시프트가 됨을 다시 음미하라.)

9.4 edge detector / pulse generator

핵심: 신호를 한 클럭 지연시켜 현재값과 비교 $\rightarrow$ 변화한 순간 1클럭 펄스. 레벨 신호를 이벤트로 바꾸는 만능 도구 (핸드셰이크, CDC 후 toggle $\rightarrow$ pulse 복원에 단골).

```
logic sig_d;
always_ff @(posedge clk) sig_d <= sig;
assign rise_pulse = sig & ~sig_d;    // 상승엣지 1클럭
assign fall_pulse = ~sig & sig_d;    // 하강엣지 1클럭
```

9.5 debouncer

핵심: 기계식 스위치/노이즈 입력이 잠깐 떨리는 것을 무시하고, **일정 시간 안정될 때만** 출력 갱신. 카운터로 "안정 유지 시간"을 센다.

```
// in이 STABLE 클럭 동안 같은 값 유지하면 그 값을 채택
always_ff @(posedge clk) begin
    in_d <= in_sync;                // (외부 비동기 입력이면 먼저 2FF 동기화)
    if (in_sync != in_d) cnt <= '0;    // 바뀌면 카운터 리셋
    else if (cnt == STABLE) out <= in_d;    // 충분히 안정  $\rightarrow$  채택
    else cnt <= cnt + 1'b1;
end
```

(외부 핀처럼 비동기인 입력은 debounce 전에 2FF synchronizer로 먼저 받아야 한다 — 05 참조.)

9.6 동기 FIFO 깊이 계산

동기 FIFO(같은 클럭의 producer/consumer 속도 차 흡수, 버스트 평탄화). **깊이 계산**의 핵심 아이디어는 "**버스트 동안 들어오는 양 - 그동안 빠지는 양**"의 최대값을 담을 만큼 깊어야 한다는 것:

필요 깊이 $\approx$ (버스트 길이 동안 write되는 데이터 수) - (그동안 read되는 데이터 수)

예: write가 매 클럭, read가 4클럭당 1개씩, 버스트 100개라면 약 $100 - 25 = 75$ 깊이 필요(여유 포함). 동기 FIFO는 read/write 포인터가 같은 클럭이라 비교가 쉽다. **다른 클럭이면 async FIFO + gray-code 포인터 + 동기화가 필요하고** 깊이 계산에 동기화 latency까지 더해야 한다(자세히는 05 참조).

9.7 arbiter — 고정 우선순위 / round-robin

여러 요청자(requester)가 하나의 자원(버스, 메모리 포트)을 다룰 때 누구에게 grant를 줄지 결정.

- **고정 우선순위(fixed priority)**: 항상 정해진 순서로. 구현 단순(priority encoder)하지만 낮은 우선순위는 starvation(굶주림) 위험.

```
// 4-요청 고정 우선순위: req[0]이 최우선
always_comb begin
    grant = '0;
    if (req[0]) grant[0] = 1'b1;
    else if (req[1]) grant[1] = 1'b1;
    else if (req[2]) grant[2] = 1'b1;
    else if (req[3]) grant[3] = 1'b1;
end
```

- **round-robin**: 마지막에 grant 받은 다음 순번부터 다시 검사 → **공평**, starvation 없음. "포인터(마지막 grant 위치)를 들고, 거기서부터 회전하며 첫 req를 찾는" 구조. 메모리 컨트롤러의 뱅크/요청 스케줄링에 단골.

핵심 아이디어만 잡으면(우선순위 인코더 + 회전 포인터) 즉석에서 변형 가능하다. 면접에서 "arbiter 짜보라"는 단골이므로 round-robin의 "포인터 회전" 개념을 말로 설명할 수 있어야 한다.

🧠 메모리: 위 블록 대부분이 메모리 컨트롤러에 그대로 등장한다 — refresh 카운터, command 시퀀스 shift, 뱅크 arbiter, write/read FIFO. "메모리 디바이스 디테일은 입사 후 채우겠지만, 컨트롤러/Peri의 디지털 빌딩블록은 이미 손에 있다"가 캡 방어의 구체적 근거다.

10. RTL → 합성 → 게이트 흐름과 PPA, 메모리 컴파일러

10.1 합성이 하는 일 — "기능 동일한 다른 구조로 매핑"

합성(synthesis)은 RTL을 받아 **기능적으로 동일한** 게이트 네트리스트로 변환한다. 핵심은 "**동일한 기능을 여러 구조로** 만들 수 있고, 도구가 제약에 맞춰 그 중 하나를 고른다"는 것이다. 예: `a+b+c+d`를 직렬 덧셈기 체인으로도,

트리 구조로도 만들 수 있는데, 빠른 클럭이 필요하면 트리(지연↓·면적↑)를, 면적이 중요하면 체인을 고른다.

합성을 모는 **제약(constraints, SDC)**: - 클럭: 목표 주파수(타이밍 제약). 06 참조. - **면적(area)**: 게이트 수 한도. - **전력(power)**: 클럭 게이팅, 멀티 Vt 셀 선택 등.

10.2 PPA trade-off

PPA = **Power·Performance·Area**. 셋은 서로 당긴다: 빠르게(Performance↑) 하려면 큰 셀·깊은 파이프 (Area·Power↑), 작게(Area↓) 하려면 느려지거나 경로가 깊어진다. 설계자의 일은 "주어진 제약 안에서 이 셋의 최적 균형을 찾는 것"이다. 나는 학부 AES에서 이걸 직접 했다 — S-box를 256B LUT(메모리·면적 큼) 대신 **composite-field logic**으로 재구현해 면적을 27,000→10,000 GE로 63% 줄였고(지연은 약간 손해 보는 trade-off), critical path의 불필요 MUX도 제거했다(04_resume 9번). **이 PPA 최적화 경험이 내가 설계 방법론으로 진로를 잡은 출발점이다.**

10.3 메모리 컴파일러와 SRAM (6T bitcell, .lib/.lef)

메모리는 표준셀로 만들지 않는다. **메모리 컴파일러(memory compiler)**가 원하는 깊이·폭의 SRAM 매크로를 자동 생성한다.

- **6T bitcell**: SRAM 한 비트 = 교차결합 인버터 2개(4T) + 액세스 트랜지스터 2개 = **6 트랜지스터**. 정적(전원 유지 시 값 보존), 빠름. (DRAM은 1T1C — 트랜지스터 1 + 커패시터 1, 작지만 누설 → refresh 필요. 02_DRAM 참조.)
- 컴파일러는 매크로와 함께 **.lib(타이밍·전력 모델)**, **.lef(물리 abstract: 크기·핀 위치·블로키지)**, 시뮬레이션 모델을 내준다. 디지털 설계자는 이 **hard macro**를 인스턴스화하고 주변 **peri 로직(주소 디코드, 데이터 정렬, BIST)**을 RTL로 붙인다.

면접 골격(Q "SRAM/메모리 컴파일러?"): "메모리는 표준셀이 아니라 메모리 컴파일러가 6T bitcell 기반으로 원하는 크기의 SRAM 매크로와 .lib/.lef를 생성합니다. 설계자는 이 hard macro를 인스턴스화하고 peri 로직·MBIST를 붙입니다. DRAM은 같은 'compiled macro + peri 디지털' 구조지만 cell이 1T1C라 refresh가 추가됩니다." ← 메모리 갭을 디지털 설계 언어로 메우는 핵심 그림.

🧠 메모리 브릿지: SRAM은 학부 때 **주도적으로 직접 설계**했다(자소서 근거). DRAM은 cell만 다를 뿐 "컴파일된 array + peri 디지털 + BIST" 구조가 동일하므로, 내 SRAM/Peri 설계 경험이 메모리로 자연 확장된다.

11. 면접 연결 — 골격 + "직접 설계 가능" 브릿지

각 질문에 **30초(핵심 한 줄)**와 **1분(원리+예시)** 골격을, 그리고 마지막에 갭 방어 브릿지를 둔다.

11.1 "동기 vs 비동기 설계, 왜 동기가 주류?"

- **30초**: 동기는 모든 FF가 공통 클럭이라 STA로 setup/hold를 **정적 검증**할 수 있어 예측 가능합니다. 비동기는 빠를 수 있으나 race-hazard가 많고 정적 검증이 약해, ASIC/메모리는 동기가 주류이고 도메인 경계만 CDC로 다룹니다.

- 1분: + "동기가 이긴 이유는 '빠르다'가 아니라 '검증 가능하다'입니다. 칩은 동기 섬들 + 비동기 다리(CDC) 구조죠."

11.2 "Moore vs Mealy"

- 30초: Moore는 출력이 상태만의 함수라 glitch 없이 안정적이나 1클럭 느리고, Mealy는 상태+입력이라 빠르고 상태가 적지만 입력 glitch가 출력에 셉니다. 보통 출력을 한 번 더 register해 절충합니다(registered Mealy).
- 1분: + "1011 검출기처럼 overlap을 살리려면 상태도를 그려 next/output을 2-block으로 짍니다. encoding은 면적이면 binary, 속도면 one-hot, 도메인 횡단이면 gray입니다."

11.3 "blocking vs non-blocking"

- 30초: 순차(클럭드)에는 non-blocking $\leq$, 조합에는 blocking $=$ 을 씁니다. $\leq$ 는 우변을 동시에 평가·동시에 대입해 진짜 시프트·race-free FF가 되고, $=$ 는 순차 실행이라 조합 계산에 맞습니다. 섞으면 sim-synth mismatch·race가 납니다.
- 1분: + 시프트 레지스터 예시로 " $\leq$ 라야 q2가 직전 q1을 받아 3단 시프트가 된다"를 설명.

11.4 "파이프라인이 개선하는 것"

- 30초: 긴 조합 경로를 레지스터로 쪼개 stage를 짧게 만들어 클럭 주파수=throughput을 올립니다. 하나의 latency는 유지/증가하지만 단위시간당 처리량이 늘죠.
- 1분: + "stage 균형(병목 stage가 fmax 결정)·retiming이 핵심이고, 오버헤드·latency·해저드 penalty 때문에 무한정 깊게는 못 쪼개 최적 깊이가 있습니다. 해저드는 forwarding·stall·branch prediction으로 푼다 — 5-stage MIPS에서 직접 구현했습니다."

11.5 "sim-synth mismatch 원인"

- 30초: incomplete sensitivity list, 의도치 않은 latch 추론, blocking/non-blocking 오용, 비합성 구문 (initial·delay), X 처리 차이입니다. lint+CDC+LEC로 차단합니다.
- 1분: + "저는 LEC(Formality)로 RTL $\equiv$ netlist를 수천만 line 규모로 sign-off했고, 이 mismatch가 왜 생기는지 게이트 레벨까지 추적해온 사람이라 처음부터 합성 친화 RTL을 짍니다."

11.6 "reset synchronizer 왜 필요한가"

- 30초: async reset은 즉시 초기화엔 좋지만 deassert가 클럭 엣지 근처면 recovery/removal 위반으로 메타가 납니다. assert는 비동기 즉시, deassert는 2FF로 동기화해 모든 FF가 같은 엣지에 풀리게 하는 게 reset synchronizer입니다.
- 1분: + "reset도 신호라 도메인을 건너면 RDC입니다. 저는 RDC false violation을 부분집합 매트릭스 논리로 87.7% 줄인 방법론을 만들었습니다(04_resume 1번)."

11.7 핵심 브릿지 — "방법론만이 아니라 직접 설계 가능"

면접관이 "검증/방법론만 한 거 아니냐"고 압박하면, 다음 논리로 정면 돌파한다:

"오히려 반대로 보셔야 합니다. 저는 학부에서 5-stage MIPS 파이프라인, SoC, VLSI를 직접 설계했고 SRAM을 주도적으로 설계했으며, AES를 RTL로 구현하며 S-box를 logic으로 재구현해 면적을 63% 줄이는

PPA 최적화를 했습니다. 그리고 4년간 sign-off를 하며 **합성 후 무엇이 깨지는지** (sensitivity·latch·blocking·CDC·RDC·타이밍)를 게이트 레벨까지 추적했습니다. **합성 후 깨지는 이유를 아는 사람이 가장 안 깨지는 RTL을 짭니다.** 메모리 디바이스 디테일은 입사 후 빠르게 채우겠지만, 디지털 설계·합성·검증의 문제는 메모리도 동일하고, 거기엔 제가 이미 깊이 들어가 있습니다."

이 브릿지의 힘은 검증 경험을 약점이 아니라 설계 깊이의 증거로 뒤집는 데 있다.

12. 스스로 점검 (정독 후 막힘 없이 답하면 합격선)

1. "RTL은 동작을 레지스터 사이 조합논리로 기술한다"를 한 문장으로 설명하고, 누산기 코드 5줄을 보고 머릿속 회로(레지스터·덧셈기·피드백)를 그릴 수 있는가?
2. 조합 vs 순차의 차이를 "메모리 유무"로 설명하고, 동기 설계가 주류인 이유를 **"STA로 정적 검증 가능"** 한 줄로 답할 수 있는가?
3. `always_ff` / `always_comb` 를 언제 쓰는지, 둘을 섞지 않는 이유를 말할 수 있는가?
4. blocking과 non-blocking의 차이를 **시프트 레지스터 예시**로 설명하고, 왜 FF엔 `<=`, 조합엔 `=`인지 race 관점에서 답할 수 있는가?
5. 의도치 않은 latch가 생기는 조건(분기 대입 누락)과 방지법(모든 경로 대입/기본값)을 코드로 보일 수 있는가?
6. sim-synth mismatch의 5가지 원인을 대고, 그것을 막는 sign-off(lint/CDC/LEC)와 자기 경험을 연결할 수 있는가?
7. Moore vs Mealy를 출력 의존성·latency·glitch로 비교하고, registered Mealy 절충과 state encoding 3종 선택 기준을 말할 수 있는가?
8. "1011" overlap 검출기의 상태도를 그리고 2-block FSM 코드로 옮길 수 있는가? safe FSM(default 복구)의 필요성은?
9. 파이프라인이 throughput를 올리고 latency는 유지/증가시키는 이유를 세탁 비유로 설명하고, "무한정 깊게 못 쪼개는" 3가지 한계를 덜 수 있는가?
10. 해저드 3종(구조/데이터(RAW·read-after-write, WAR·write-after-read name)/제어)과 해결책(forwarding/stall·bubble/branch prediction)을 MIPS 경험과 연결할 수 있는가?
11. async vs sync reset의 trade-off와 **reset synchronizer(async assert + sync deassert)**가 필요한 이유, RDC 연결을 한 호출에 말할 수 있는가?
12. glitch가 FF의 D엔 무해하지만 clock/reset/CDC enable엔 치명적인 이유와, 빌딩블록(edge detector÷홀수 분주·round-robin arbiter·동기 FIFO 깊이)을 핵심 아이디어로 즉석 설명/스케치할 수 있는가? 그리고 마지막에 **"방법론만이 아니라 직접 설계 가능"** 브릿지를 자소서 근거(MIPS·SRAM·AES)와 함께 30초로 칠 수 있는가?

연계: 메타스터빌리티·CDC·RDC·동기화 회로 → [concepts/05_CDC_RDC_완전이해.md](#) /
setup·hold·recovery·removal·타이밍 수식 → [concepts/06_STA_완전이해.md](#) / 카드 요약 →
[daily_study/6_digital.md](#) / 자소서 근거·꼬리질문 방어 → [04_resume_selfmastery.md](#).